



Securing FPGAs in IoT: a new run-time monitoring technique against hardware Trojan

Jian Cheng¹ · Quanyuan Feng¹ · Chao Li² · Wenxiu Yang¹

Accepted: 16 February 2023 / Published online: 11 March 2023

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

In this paper, we propose a new run-time monitoring technique to detect hardware Trojan (HT) on working field-programmable gate array (FPGA), which is helpful for reinforcing the hardware security of IoT devices. First, we place multiple temperature sensors on FPGA and collect their value at each time interval. Then, we construct a predictive model which can estimate the distribution of sensor frequency in real-time. If the difference between the predicted value and the real one surpasses a dynamically updated threshold, an HT activation is reported. The experiment results reveal that our technique has high detection accuracy ($\geq 98\%$), low false negative ratio ($\leq 1.5\%$) and zero false positive ratio under various FPGA HT benchmarks.

Keywords IoT security · Integrated circuit · Hardware Trojan detection · Thermal side channel

1 Introduction

Nowadays, Internet of Things (IoT), which bridges myriads of devices via Internet connection, becomes the infrastructure of many modern applications, such as smart city, smart grid and Industry 4.0, etc. [1–4]. To satisfy different requirements from various IoT applications, field-programmable gate arrays (FPGAs), which is enable true flexibility and scalability with inherent hardware programmability, are widely used in many IoT devices, as shown in Fig. 1 [5–7]. For example, ZTE Star-2 smartphone is powered by Lattice Semiconductor's iCE40 FPGA to perform Infrared Radiation (IR) remote control task; in smart home application, DELL C4140 server integrates an Intel Stratix-10 FPGA as an accelerator for fast IoT data processing; in smart medical area, medical endoscope from Lighthouse Imaging Corporation utilizes a

Xilinx ZNQY-7 FPGA to improve its image acquisition and processing capabilities. In addition, FPGA are also widely used in the field of artificial intelligence, smart Manufacturing, medical electronics and image processing [8–21].

As the popularity of FPGA in IoT grows, more and more adversities try to utilize the vulnerabilities in FPGA to attack IoT space [22]. As recently studied, even though an encrypted configuration bitstream of FPGA can be cracked, which paves the way to reversing FPGA circuit and implant hardware Trojan (HT) into FPGA [23]. HT is a malicious modification to the original circuitry in FPGA, and once activated, it could perform malicious function or steal sensitive information. The present of HT poses great threats on IoT security. To tackle this issue, in this paper, we propose a new run-time monitoring technique which can protect FPGAs against HTs attacks, reinforcing the hardware security for IoT.

The remaining parts of the paper are organized as follows. We introduce related work in Sect. 2 and detail our method in Sect. 3. In Sect. 4, we perform extensive experiments to validate our approach. Finally, Sect. 5 concludes the paper.

✉ Jian Cheng
chengjian510786@my.swjtu.edu.cn

✉ Chao Li
cynthia0217@163.com

¹ The School of Information Science and Technology, Southwest Jiaotong University, Chengdu 610031, China

² Zhijiang College of Zhejiang University of Technology, Shaoxing 312030, China

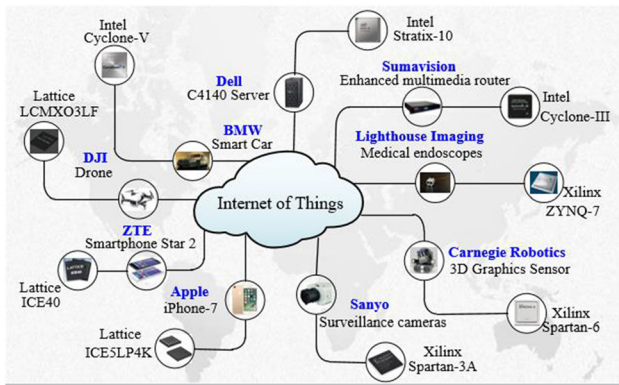


Fig. 1 Exemplary application of FPGA in IoT world

2 Related work

According to the stage of FPGA lifecycle, HT detection methods can be roughly classified to three categories, i.e., design-time, test-time and run-time.

1. According to the stage of FPGA lifecycle, HT detection methods can be roughly classified to three categories, i.e., design-time, test-time and run-time.
2. Design-time Technique: It protects FPGAs in early stages by analyzing their Hardware Description Language (HDL) code [24] and/or gate-level netlists [25]. Many techniques have been presented to verify the trustworthiness of code/netlist, such as netlists static analysis [26], unused logic identification [27], structural feature analysis [28], boolean functional analysis [29] and state transition probability analysis [30], etc. The design-time techniques can detect various HTs with high accuracy. However, all of them need the original chip design files, which may not always be available for security engineers.
3. Test-time Technique: The test-time technique touches upon three folds, (1) functional testing, (2) reverse engineering, and (3) side-channel analyzing. Functional testing approaches intend to find out HTs which change the functionality of FPGAs when activated [31, 32]. The reverse engineering and side-channel analyzing techniques for HT detection are also presented to detect HTs. For example, Zhang et al., develop the first precise FPGA reverse engineering tool and use it for HT identification [33]. In side-channel analysis domain, electromagnetic radiation and thermal side channel are often utilized for FPGA HT detection [34–36]. Note that these test-time techniques are quite time-consuming. Hence, they perform well on catching HTs from small scale chips rather than large scale chips.
4. Run-time Technique: The run-time approaches are considered as the last mile of defense against hardware

Trojan. For example, the authors in [37] propose a method which makes use of on-chip sensors to track the deviations of thermal profiles against golden chip, detecting the activated HTs quickly [38]. In [39], Zhao et al. propose a method based on the chaos theory. With power traces information, it can detect HT with high accuracy. Although the run-time techniques can quickly detect the activation of HTs, the existing techniques depend on golden chips which are hard to be obtained.

In this paper, we propose a run-time low-overhead HT monitoring technique. In contrast to the existing approaches, our method can detect HTs without a golden chip, thus it is easier to be integrated onto FPGA for on-line HT detection.

3 Our HT monitoring method

As shown in Fig. 2, our hardware Trojan monitoring method consists of five steps. (1) We construct multiple ROs (Ring-Oscillators), each of which is chained with odd number of inverters. After that, we implant them into FPGA to form a sensor network with mesh topology. (2) We measure the frequency of sensors, and denoise the raw readouts through two operations, i.e., outlier removal and mean filtering. (3) We set up a predictive model to estimate the sensor frequency. After initializing error covariance matrix, process covariance matrix and measurement covariance matrix, we deduce the expression for state transition matrix and control-input matrix. (4) We construct an estimation error matrix to drive a Kalman filter. Thereafter, we obtain the difference of sensor frequency between real and estimated values. (5) By using a residual autocorrelation method, we can detect the thermal fluctuation caused by the activation of HT. Once the fluctuation triggers a dynamically updated threshold, we report that there exists an activated HT in FPGA.

3.1 Sensor network design and layout

In this step, we make use of Ring-Oscillators (ROs) as on-chip temperature sensors, and construct a sensor network with mesh topology. Once an HT is activated, the temperature of nearby region will rise, leading to the increasing of RO (Ring Oscillator) frequency. Hence, many researches make use of RO as temperature sensor. They implant ROs into FPGA to monitor the fluctuation of chip temperature through sensors frequency [40, 41].

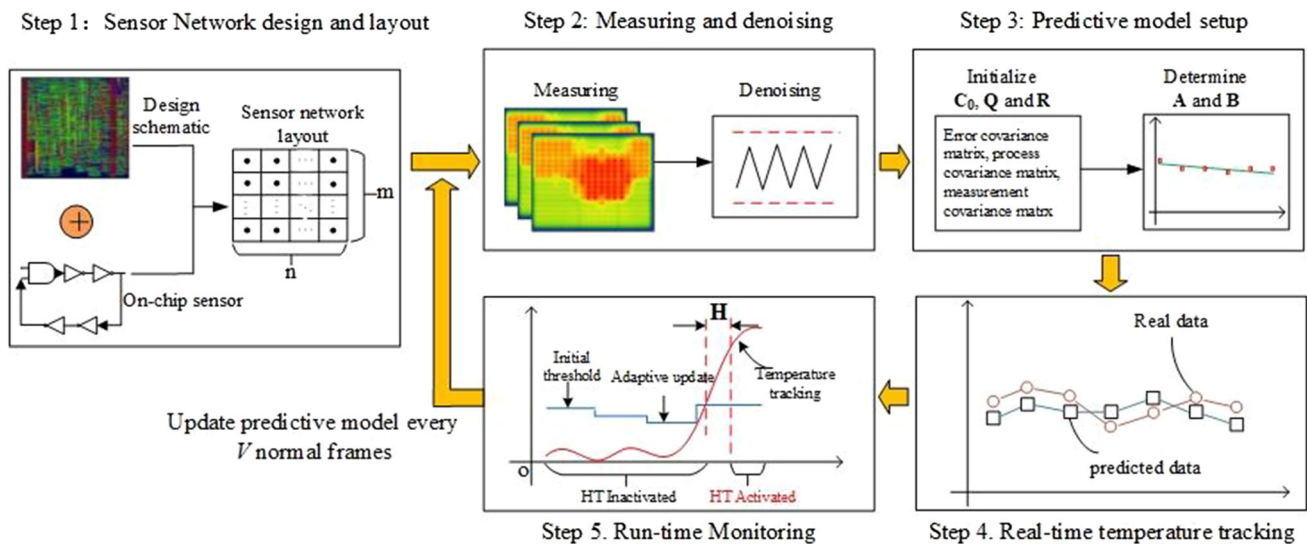


Fig. 2 The proposed run-time HT monitoring method

3.2 Measuring and denoising

In this step, we first gather the frequency values for N ROs in the sensor network, and denote them by using an N -dimension vector $\mathbf{F}_k$. Here k is an integer representing the number of sample time. Since the raw readouts from chip are often contaminated by noises and glitches, we perform a denoising process consisting outlier removal and mean filtering on $\mathbf{F}_k$. The progress of our denoising process are shown in Fig. 3.

Outlier removal: The variation n range of RO frequency $[f_{\min}, f_{\max}]$ is obtained. Any frequency below or above this range is recognized as an outlier, and is removed. Here $f_{\min}$ and $f_{\max}$ are calculated by using the Gaussian-based method as mentioned in [42]. In detail, we obtain the first quartile (f_1) and the third quartile (f_3) according to the raw readouts, then calculate the value of $f_{\min}$ and $f_{\max}$ by using formula (1)

$$\begin{cases} f_{\min} = f_1 - 1.5(f_3 - f_1) \\ f_{\max} = f_3 + 1.5(f_3 - f_1) \end{cases} \quad (1)$$

Mean filtering: For a given number k , mean filtering calculate the average value from M continuous vectors

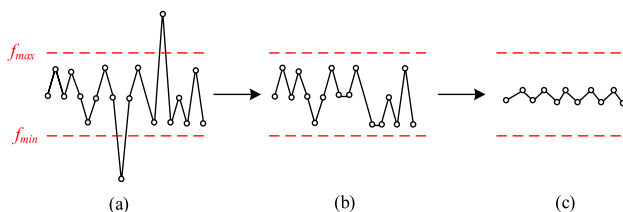


Fig. 3 The denoising process of sensor readout: (a) raw data (b) after outlier removal (c) after mean filtering

$$[\mathbf{F}_{k-M+1}, \dots, \mathbf{F}_{k-2}, \mathbf{F}_{k-1}, \mathbf{F}_k]$$

and store it into another vector $\mathbf{T}_M$.

The threshold $f_{\min}$ and $f_{\max}$ for denoising operation are set to be 25 MHz and 75 MHz, respectively. Note that the value of M should be chosen carefully. Because a small M is not sufficient for denoising, whereas a large M smooths out the fluctuation introduced by trojans. In this paper, we set M to be 100 as advised in [43].

3.3 Predictive model setup

In this step, a predictive model based on Kalman filtering is setup for real-time frequency estimating. First, the error covariance matrix ($\mathbf{C}_0$), the process covariance matrix ($\mathbf{Q}$) and measurement covariance matrix ($\mathbf{R}$) are initialized. Here $\mathbf{C}_0$ is a $N \times N$ matrix, its elements are separately set to be random values between 0.01 and 0.1. $\mathbf{Q}$ is initialized to be $0.1 \times \mathbf{I}$, where $\mathbf{I}$ is an $N \times N$ identity matrix. The initial value of $\mathbf{R}$ is drawn from a normal distribution $N(0, 0.1)$.

Next, we determine the state transition matrix $\mathbf{A}$ and the control-input matrix $\mathbf{B}$. By using three consecutive vectors $\mathbf{T}_{k-1}$, $\mathbf{T}_k$ and $\mathbf{T}_{k+1}$, the two matrixes can be expressed as follows.

$$\mathbf{T}_k = \mathbf{A}\mathbf{T}_{k-1} + \mathbf{B} \quad (2)$$

$$\mathbf{T}_{k+1} = \mathbf{A}\mathbf{T}_k + \mathbf{B} \quad (3)$$

Subtracting formula (2) from (3), the state transition matrix $\mathbf{A}$ can be written as:

$$\mathbf{A} = (\mathbf{T}_{k+1} - \mathbf{T}_k)(\mathbf{T}_k - \mathbf{T}_{k-1})_R^{-1} \quad (4)$$

where $(\bullet)_R^{-1}$ denotes the right inverse of a matrix.

Afterwards, we substitute formula (4) into (2), the control-input matrix $\mathbf{B}$ can be represented as follows.

$$\mathbf{B} = \mathbf{T}_k - (\mathbf{T}_{k+1} - \mathbf{T}_k)(\mathbf{T}_k - \mathbf{T}_{k-1})_R^{-1} \mathbf{T}_{k-1} \quad (5)$$

3.4 Real-time frequency estimating

After setting up predictive model, we can estimate the sensors frequency by using a Kalman filtering. In detail, we use frequency features at time $k-1$ ($\mathbf{T}_{k-1}$) to predict the frequency features of next time ($\mathbf{T}_k$). The predictive equation can be written as:

$$\mathbf{T}[k|k-1] = \mathbf{A}\mathbf{T}[k-1|k-1] + \mathbf{B} \quad (6)$$

where $\mathbf{T}[k-1|k-1]$ is the updated frequency feature matrix at time $k-1$, whose initial value is $\mathbf{T}_1$. $\mathbf{T}[k|k-1]$ is the predicted frequency feature matrix of time k based on the observations value at time $k-1$. The covariance matrix standing for the predicted error is calculated using formula (7).

$$\mathbf{C}[k|k-1] = \mathbf{A}\mathbf{C}[k-1|k-1]\mathbf{A}^T + \mathbf{Q} \quad (7)$$

where $\mathbf{C}[k-1|k-1]$ is the updated error covariance matrix of time $k-1$, and $\mathbf{C}[k|k-1]$ is the predicted error covariance matrix of time k based on the observations at time $k-1$. So far, the gain of Kalman filter can be evaluated by using formula (8).

$$\mathbf{K}[k] = \mathbf{C}[k|k-1](\mathbf{R} + \mathbf{C}[k|k-1])^{-1} \quad (8)$$

Then, the temperature feature matrix can be updated by using formula (9).

$$\mathbf{T}[k|k] = \mathbf{T}[k|k-1] + \mathbf{K}[k]\mathbf{e}[k] \quad (9)$$

where $\mathbf{T}[k|k]$ is the updated temperature feature matrix of time k , and $\mathbf{e}[k]$ is the estimation error matrix of time k .

$$\mathbf{e}[k] = \mathbf{T}_k - \mathbf{T}[k|k-1] \quad (10)$$

Finally, we can update the error covariance matrix by using formula (11).

$$\mathbf{C}[k|k] = (\mathbf{I} - \mathbf{K}[k])\mathbf{C}[k|k-1] \quad (11)$$

3.5 Run-time HT monitoring

In this step, the frequency fluctuation of sensors is augmented by using residual autocorrelation method, and thereafter, we justify the existence of HTs with a dynamic threshold.

Given S consecutive estimation error matrixes $\mathbf{e}[k-S], \mathbf{e}[k-S+1], \dots, \mathbf{e}[k]$, the residual autocorrelation of time k , denoted as $\mathbf{a}[k]$, can be calculated by formula (12).

$$\mathbf{a}[k] = \frac{1}{S} \sum_{i=k-S+1}^k (\mathbf{e}[i]\mathbf{e}[i-1]^T) \quad (12)$$

Then, the monitoring index of time k which is the Frobenius-norm of $\mathbf{a}[k]$ and denoted as $\mathbf{M}[k]$, can be obtained.

$$\mathbf{M}[k] = \|\mathbf{a}[k]\|_F = \sqrt{\sum_{i=1}^N \sum_{j=1}^N |a_{ij}^k|^2} \quad (13)$$

where a_{ij}^k is the element in the i -th row and j -th column of $\mathbf{a}[k]$.

Now, we justify whether a FPGA contains HTs according to the inequation (13). If it is satisfied for a certain number of continuous vectors, we will alarm there might be an activated HTs in FPGA. Otherwise, a none-HT state is reported.

$$\mathbf{M}[k] > a_T \quad (14)$$

where a_T is the dynamic threshold, which is updated cyclically during the run-time monitoring process. For a certain value k , if we do not detect the activation of HT during the previous V time slots, the threshold a_T will be updated with formula (15).

$$a_T = b\mu + a_{\max} \quad (15)$$

where b is the tuning parameter with default value of 1, μ is the mean value of $\mathbf{M}[i]$ and $a_{\max}$ is the max value of $\mathbf{M}[i]$. Here $i = k-V, k-V+1, \dots, k-1$.

4 Experiments

4.1 Experimental environment

As shown in Fig. 4, our experimental hardware environment consist of a computer and a development board. First, we download the bitstream, which contains the test circuit, the RO temperature sensor network and the HTs, into the development board through the JTAG interface. Then, the frequency of each RO is upload to the computer through COM interface. HTs are triggered by press a key on the

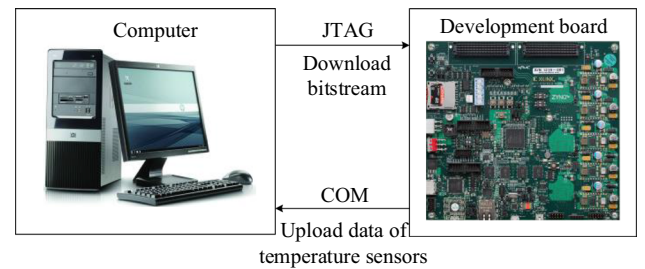


Fig. 4 The structure of our experimental hardware environment

Table 1 HT benchmarks used in our experiment

HT Benchmark	Type	Functionality	Power change
BasicRSA-T200	Combinational	DoS	25.00%
BasicRSA-T400	Sequential	Information leakage	14.29%
AES-T1800	Sequential	DoS	9.79%
BasicRSA-T100	Combinational	Information leakage	7.69%
AES-T1900	Sequential	DoS	5.77%
RS232-T900	Combinational	DoS	2.32%
AES-T700	Sequential	Information leakage	1.88%
RS232-T400	Combinational	Information leakage	1.22%

Table 2 Experiment result of our proposal

HT Benchmark	ACC (%)	FN (%)	FP(%)	RT(s)
BasicRSA-T200	99.0	1.0	0.0	0.5
BasicRSA-T400	98.5	1.5	0.0	0.8
AES-T1800	99.0	1.0	0.0	0.8
BasicRSA-T100	99.0	1.0	0.0	1.1
AES-T1900	98.5	1.5	0.0	1.2
RS232-T900	99.5	0.5	0.0	1.8
AES-T700	98.5	1.5	0.0	2.0
RS232-T400	99.0	1.0	0.0	2.4

development board. By using our predictive model, the HT activation will be detected if the difference between the predicted value and the real one surpasses a dynamically updated threshold.

To evaluate the effectiveness of our method, we make extensive experiments on Xilinx FPGA Zynq-7000. Eight circuitries with different function are chosen from Trust-Hub benchmarks. To observe the influence of activated HT on power consumption, we conduct simulations in Vivado for all HT benchmarks under 100 MHz clock rate. The variation of power between circuit with inactivated and activated HTs are shown in Table 1. We implant multiple sensors into FPGA with mesh topology to cover the region of test circuits. To ensure the temperature variation caused by HTs can be detected, we set the separation of ROs to be 7 slices, as advised in [44]. In addition, we started to

measure the ROs frequency once the chip is powered ON, and the frequency value is memorized at every 0.1 s.

To train the predictive model, we track the sensors frequency for 60 min while HT stays dormant during the whole time. Note that the inactivated HT consumes little power which is almost negligible. Hence, we take the frequency of sensors in FPGA with an inactive HT as an HT-free FPGA.

Thereafter, we monitor the FPGA for 60 min. An activation instruction is randomly generated during the monitoring time, and the process repeats for 100 times to count the precision of our HT detection method.

4.2 HT monitoring results

The accuracy rate (ACC), false positive (FP), false negative (FN) and response time (RT) are gained to evaluate the performance of our approach. As shown in Table 2, our HT monitoring method successfully detects all HT activations with high probability. The overall ACC is above 98% with little FN (i.e., 0.5–1.5%) and zero FP. Our method can find the activation of HT within several seconds (i.e., 0.5–2.4 s), and the larger the power changes after HT's activation, the quicker detection is made.

4.3 Comparison to other methods

We compare our method with three state-of-the-art HT detection techniques, i.e., Current Mirror [45] (CM), Temperature Tracking [38] (TT) and Chaos Theory [39]

Table 3 The comparison of application requirement

Method	Golden Chip	Hardware overheads	Ease of application	Side-channel
Ours	Not required	N ROs, 1 counter	Easy	Power
CM	Required	N current mirrors, N current comparator, N scan chain registers	Easy	Path delay
TT	Required	N ROs, N counters	Easy	Power
CT	Not required	N ROs, N counters	Hard	Power

(CT) in aspects of application requirement and performance.

The application requirement of different methods are summarized in Table 3. As we can see, both CM and TT methods require the golden chip of the design, which is not always available. On the other hand, our method as well as CT method do not depend on the golden chip, resulting in a broader application scenario.

As for the aspect of hardware overheads, all four methods need additional hardware resources. However, the implementation of RO sensor consumes less resource than that of current sensor since it has to integrate some mirrors and registers. Hence, TT, CT and our method require less resources than CM. Moreover, we should note that our temperature sensors require only one counter to number the rising edge of RO's output pulse, which further reduces the hardware overheads compared to TT and CT methods. From the degree on ease of application, CT method is the hardest one to be applied, since it not only requires thermal RC modeling of IC, but also needs the nonlinear dynamic power dissipation of ICs in reconstructed phase space. Finally, our method and CM are based on frequency and path delay side-channel, respectively, whereas TT and CT methods detect HTs using information of power side-channel.

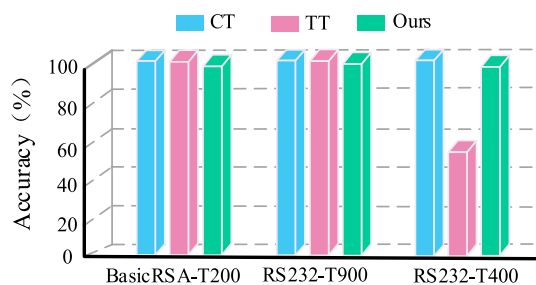
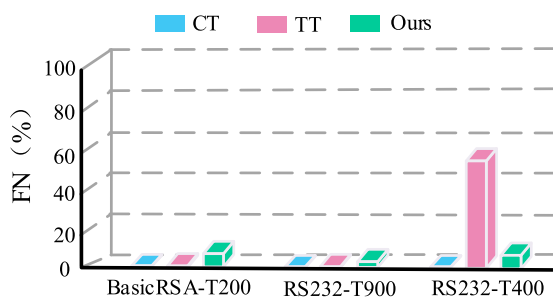
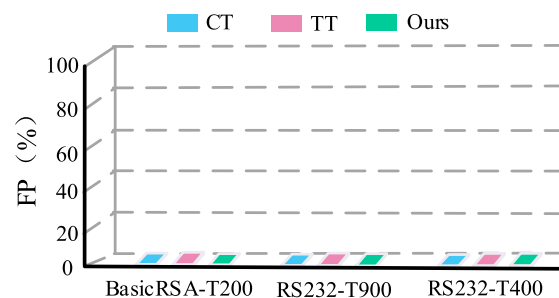


Fig. 5 Accuracy comparison between three methods



(a) FN comparison



(b) FP comparison

Fig. 6 FAR and MR comparison between three methods

The comparison of method performance is displayed in Fig. 5. Three HT benchmarks, BasicRSA-T200, RS232-T900 and RS232-T400, are chosen for performance comparison. The results of TT and CT method are from [39] and [40]. As shown in Fig. 4, CT method performs the best on HT activation monitoring. Its accuracy is 100% for all benchmarks. On the other hand, we can observe that the accuracy of TT method is 100% under BasicRSA-T200 and RS232-T900 benchmarks, however, it drops dramatically to 50% under RS232-T400 benchmark. Our method achieves a steady accuracy above 98% in all benchmarks, which is better than the TT method.

The comparison of FN between three methods is given in Fig. 6a. Our method raises small amount of false positive under BasicRSA-T200 and RS232-T900 benchmarks, whereas TT and CT methods have none. Besides, under RS232-T400 benchmark, CT method has no false positive alarm as well, while our method suffices a small FN that is far less than TT method (almost 50%). Note that the FP results for the three methods are all 0% in Fig. 6b, meaning that all three methods can detect the activated HT in the benchmark.

According to the above mentioned results, we can draw the following conclusions.

1. CT method has the best detection performance with 100% accuracy, 0% FN and 0% FP. However, this method is hard to be applied on FPGA due to its requires complex model.
2. TT method suffers a challenge on uneven detection performance. For some benchmarks, such as RS232-T400, the accuracy drops drastically to 50% due to its fixed threshold setting.
3. Our method gain high HT detection accuracy, and it exhibits a little FN caused by sensor noise. Meanwhile, our method requires the least amount of hardware resource, which makes it more practical and easier to be implemented on FPGA.

5 Conclusion

This paper focuses on the HT threat on FPGAs in IoT devices. To tackle this issue, we propose a run-time HT monitoring technique to detect the activation of HT in FPGA. In our method, on-chip sensor is first designed and implanted to FPGA, then their frequency information is measured and denoised. After setting-up a predictive model, we can perform frequency estimating in run-time, and then compare the estimated value with real one to detect the activation of HT.

Finally, we validate our method with 8 benchmarks. The experimental results show that the detection accuracy of our method is above 98%. Comparative analysis is made between our approach and three state-of-the-art HT detection methods, revealing the strength and limitation of our method.

References

1. Lee, I., & Lee, K. (2015). The Internet of Things (IoT): Applications, investments, and challenges for enterprises. *Business Horizons*, 58(4), 431–440. <https://doi.org/10.1016/j.bushor.2015.03.008>
2. Gong, W., Zhang, X., Chen, Y., He, Q., Beheshti, A., Xu, X., Yan, C., & Qi, L., (2022). DAWAR: Diversity-aware web APIs recommendation with compatibility guarantee for mashup creation. In *The 45th international ACM SIGIR conference on research and development in information retrieval* (pp. 395–404). <https://doi.org/10.1145/3477495.3531962>.
3. Gong, W., Zhang, W., Bilal, M., Chen, Y., Xu, X., & Wang, W. (2022). Efficient web APIs recommendation with privacy-preservation for mobile app development in industry 4.0. *IEEE Transactions on Industrial Informatics*, 18(9), 6379–6387. <https://doi.org/10.1109/TII.2021.3133614>
4. Gong, W., Lv, C., Duan, Y., Liu, Z., Khosravi, M. R., Qi, L., & Dou, W. (2021). Keywords-driven web APIs group recommendation for automatic app service creation process. *Software Practice and Experience*, 51, 2337–2354. <https://doi.org/10.1002/spe.2902>
5. Oh, H., Nam, K., Jeon, S., Cho, Y., & Paek, Y. (2021). MeetGo: A trusted execution environment for remote applications on FPGA. *IEEE Access*, 9, 51313–51324. <https://doi.org/10.1109/ACCESS.2021.3069223>
6. Okada, M., Suzuki, T., Nishio, N., Waidyasooriya, H. M., & Hariyama, M. (2022). FPGA-accelerated searchable encrypted database management systems for cloud services. *IEEE Transactions on Cloud Computing*, 10(2), 1373–1385. <https://doi.org/10.1109/TCC.2020.2969655>
7. Tan, B., Mok, K., Chang, J., Lee, W., & Hwang, S. O. (2022). RISC32-LP: Low-power FPGA-based IoT sensor nodes with energy reduction program analyzer. *IEEE Internet of Things Journal*, 9(6), 4214–4228. <https://doi.org/10.1109/JIOT.2021.3103035>
8. Zhou, X., Liang, W., Yan, K., Li, W., Wang, K., Ma, J., & Jin, Q. (2022). Edge enabled two-stage scheduling based on deep reinforcement learning for Internet of Everything. *IEEE Internet of Things Journal*, Early Access. <https://doi.org/10.1109/JIOT.2022.3179231>
9. Zhou, X., Hu, Y., Wu, J., Liang, W., Ma, J., & Jin, Q. (2022). Distribution bias aware collaborative generative adversarial network for imbalanced deep learning in industrial IoT. *IEEE Transactions on Industrial Informatics*, 19(1), 570–580. <https://doi.org/10.1109/TII.2022.3170149>
10. Zhou, X., Yang, X., Ma, J., & Wang, K. (2022). Energy-efficient smart routing based on link correlation mining for wireless edge computing in IoT. *IEEE Internet of Things Journal*, 9(16), 14988–14997. <https://doi.org/10.1109/JIOT.2021.3077937>
11. Liang, W., Hu, Y., Zhou, X., Pan, Y., & Wang, K. (2022). Variational few-shot learning for microservice-oriented intrusion detection in distributed industrial IoT. *IEEE Transactions on Industrial Informatics*, 18(8), 5087–5095. <https://doi.org/10.1109/TII.2021.3116085>
12. Zhou, X., Liang, W., Li, W., Yan, K., Shimizu, S., & Wang, K. (2022). Hierarchical adversarial attacks against graph neural network based IoT network intrusion detection system. *IEEE Internet of Things Journal*, 9(12), 9310–9319. <https://doi.org/10.1109/JIOT.2021.3130434>
13. Zhou, X., Xu, X., Liang, W., Zeng, Z., Shimizu, S., Yang, L. T., & Jin, Q. (2022). Intelligent small object detection based on digital twinning for smart manufacturing in industrial CPS. *IEEE Transactions on Industrial Informatics*, 18(2), 1377–1386.
14. Lu, F., Zhang, Z., Guo, L., Chen, J., Zhu, Y., Yan, K., & Zhou, X. (2022). HFNet: A lightweight hand-crafted feature enhanced CNN for ceramic tile surface defect detection. *International Journal of Intelligent Systems*, 32(2), 10670–11069. <https://doi.org/10.1002/int.22935>
15. Chen, J., Li, Y., Guo, L., Zhou, X., Zhu, Y., He, Q., Han, H., & Feng, Q. (2022). Machine learning techniques for CT imaging diagnosis of novel coronavirus. *Neural Computing and Applications*. (Special Issue). <https://doi.org/10.1007/s00521-022-07709-0>
16. Kong, L., Li, G., Rafique, W., Shen, S., He, Q., Khosravi, M. R., Wang, R., & Qi, L. (2022). Time-aware missing healthcare data prediction based on ARIMA model. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* (Early Access). <https://doi.org/10.1109/TCBB.2022.3205064>
17. Liu, Y., Li, D., Wan, S., Wang, F., Dou, W., Xu, X., Li, S., Ma, R., & Qi, L. (2021). A long short-term memory-based model for greenhouse climate prediction. *International Journal of Intelligent Systems*, 37(1), 135–151. <https://doi.org/10.1002/int.22620>
18. Kong, L., Wang, L., Gong, W., Yan, C., Duan, Y., & Qi, L. (2021). LSH-aware multitype health data prediction with privacy preservation in edge environment. *World Wide Web Journal*, 25, 1793–1808. <https://doi.org/10.1007/s11280-021-00941-z>
19. Wang, F., Zhu, H., Srivastava, G., Li, S., Khosravi, M. R., & Qi, L. (2021). Robust collaborative filtering recommendation with user-item-trust records. *IEEE Transactions on Computational Social Systems*, 9(4), 986–996. <https://doi.org/10.1109/TCSS.2021.3064213>
20. Qi, L., Lin, W., Zhang, X., Dou, W., Xu, X., & Chen, J. (2022). A correlation graph based approach for personalized and compatible web APIs recommendation in Mobile APP development. *IEEE Transactions on Knowledge and Data Engineering* (Early Access). <https://doi.org/10.1109/TKDE.2022.3168611>
21. Chen, J., Qin, F., Lu, F., Guo, L., Li, C., Yan, K., & Zhou, X. (2022). CSPP-IQA: A multi-scale spatial pyramid pooling based approach for medical image quality assessment. *Neural Computing and Applications* (Special Issue). <https://doi.org/10.1007/s00521-022-07874-2>

22. Guo, S., Wang, J., Chen, Z., Li, Y., & Lu, Z. (2020). Securing IoT space via hardware trojan detection. *IEEE Internet of Things Journal*, 7(11), 11115–11122. <https://doi.org/10.1109/JIOT.2020.2994627>
23. Yoon, J., Seo, Y., Jang, J., Cho, M., Kim, J., Kim, H., & Kwon, T. (2018). A bitstream reverse engineering tool for FPGA hardware trojan detection. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security* (pp. 2318–2320). <https://doi.org/10.1145/3243734.3278487>.
24. Guo, X., Dutta, R. G., Jin, Y., Farahmandi F., & Mishra P. (2015). Pre-silicon security verification and validation: A formal perspective. In *Proceedings of the 52nd annual design automation conference* (pp. 1–6). <https://doi.org/10.1145/2744769.2747939>.
25. Hu, W., Mao, B., Oberg, J., & Kastner, R. (2016). Detecting hardware trojans with gate-level information-flow tracking. *Computer*, 49(8), 44–52. <https://doi.org/10.1109/MC.2016.225>
26. Huang, K., & He, Y. (2019). Trigger identification using difference-amplified controllability and dynamic transition probability for hardware trojan detection. *IEEE Transactions on Information Forensics and Security*, 15, 3387–3400. <https://doi.org/10.1109/TIFS.2019.2946044>
27. Hicks, M., Finnicum, M., King, S. T., Martin, M. M. K., & Smith, J. M. (2010). Overcoming an untrusted computing base: Detecting and removing malicious hardware automatically. In *IEEE symposium on security and privacy* (pp. 159–163). <https://doi.org/10.1109/SP.2010.18>.
28. Zhang, J., Yuan, F., Wei, L., Liu, Y., & Xu, Q. (2015). VeriTrust: Verification for hardware trust. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 34(7), 1148–1161. <https://doi.org/10.1109/TCAD.2015.2422836>
29. Waksman, A., Suozzo, M., & Sethumadhavan, S. (2013) FANCI: identification of stealthy malicious logic using boolean functional analysis. In *Proceedings of ACM SIGSAC conference on Computer & communications security* (pp. 697–708). <https://doi.org/10.1145/2508859.2516654>.
30. Zou, M., Cui, X., Shi, L., & Wu, K. (2017). Potential trigger detection for hardware trojans. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 37(7), 1384–1395. <https://doi.org/10.1109/TCAD.2017.2753201>
31. Salmani, H., Tehranipoor, M., & Plusquellic, J. (2011). A novel technique for improving hardware trojan detection and reducing trojan activation time. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 20(1), 112–125. <https://doi.org/10.1109/TVLSI.2010.2093547>
32. Nigh, C., & Orailoglu, A. (2020). Test pattern superposition to detect hardware Trojans. In *2020 design, automation & test in europe conference & exhibition (DATE)* (pp. 25–30). <https://doi.org/10.23919/DATE48585.2020.9116264>.
33. Zhang, T., Wang, J., Guo, S., & Chen, Z. (2019). A comprehensive FPGA reverse engineering tool-chain: From bitstream to RTL code. *IEEE Access*, 7, 38379–38389. <https://doi.org/10.1109/ACCESS.2019.2901949>
34. He, J., Zhao, Y., Guo, X., & Jin, Y. (2017). Hardware trojan detection through chip-free electromagnetic side-channel statistical analysis. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 25(10), 2939–2948. <https://doi.org/10.1109/TVLSI.2017.2727985>
35. Nguyen, L. N., Cheng, C., Prvulovic, M., & Zajić, A. (2019). Creating a backscattering side channel to enable detection of dormant hardware trojans. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 27(7), 1561–1574. <https://doi.org/10.1109/TVLSI.2019.2906547>
36. Pyrgas, L., Pirpilidis, F., Panayiotarou, A., & Kitsos, P. (2017). Thermal sensor based hardware Trojan detection in FPGAs. In *Euromicro conference on digital system design* (pp. 268–272). <https://doi.org/10.1109/DSD.2017.36>.
37. Forte, D., Bao, C., & Srivastava, A. (2013). Temperature tracking: An innovative run-time approach for hardware Trojan detection. *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2013, 532–535. <https://doi.org/10.1109/ICCAD.2013.6691167>
38. Bao, C., Forte, D., & Srivastava, A. (2015). Temperature tracking: Toward robust run-time detection of hardware Trojans. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 34(10), 1577–1585. <https://doi.org/10.1109/TCAD.2015.2424929>
39. Zhao, H., Kwiat, L., Kwiat, K. A., Kamhoua, C. A., & Njilla, L. (2018). Applying chaos theory for runtime hardware Trojan monitoring and detection. *IEEE Transactions on Dependable and Secure Computing*, 17(4), 716–729. <https://doi.org/10.1109/TDSC.2018.2864733>
40. Micco, L. D., Minchola, C., Leon-Franco, J. J., Boemo, E., & Antonelli, M. (2020). An annotated guide to utilize ring-oscillators as thermal sensor in FPGA technology. In *Argentine conference on electronics* (pp. 1–7). <https://doi.org/10.1109/CAE48787.2020.9046367>.
41. Franco, J. L., Boemo, E., Castillo, E., & Parrilla, L. (2010). Ring oscillators as thermal sensors in FPGAs: Experiments in low voltage. In *VI Southern programmable logic conference* (pp. 35–37). <https://doi.org/10.1109/SPL.2010.5483027>.
42. Smiti, A. (2020). A critical overview of outlier detection methods. *Computer Science Review*, 38, 100306–100308. <https://doi.org/10.1016/j.cosrev.2020.100306>
43. Yang, W. (2021). Technology of chip security monitoring based on physical field information. Master Thesis, University of Electronic Science and Technology of China, Chengdu. <https://doi.org/10.27005/d.cnki.gdzku.2021.001980>.
44. Qin, Y., & Xia, T. (2017). Sensitivity analysis of ring oscillator based hardware Trojan detection. In *17th IEEE international conference on communication technology* (pp. 1979–1981). <https://doi.org/10.1109/ICCT.2017.8359975>.
45. Cao, Y., Chang, C., Chen, S. (2013). Cluster-based distributed active current timer for hardware Trojan detection. In *IEEE international symposium on circuits and systems (ISCAS)* (pp. 1010–1013). <https://doi.org/10.1109/ISCAS.2013.6572020>.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Jian Cheng is currently pursuing the Ph.D. degree from Southwest Jiaotong University, Chengdu, China. His research interests include integrated circuits design, embedded system, wireless communications, antennas and propagation, microwave & millimeter wave technology, hardware security etc.



Chao Li is currently a lecturer at Zhijiang College of Zhejiang University of technology. She received her Ph.D. from School of Biomedical Engineering, Shanghai Jiaotong University in 2015. Her research interests covers medical image processing, Computer-Aided Diagnosis/Detection, deep learning and object detection.



Quanyuan Feng received the M.S. degree in microelectronics and solid electronics from the University of Electronic Science and Technology of China, Chengdu, China, in 1991, and the Ph.D. degree in EM field and microwave technology from Southwest Jiaotong University, Chengdu, China, in 2000. His research interests include integrated circuits design, RFID technology, embedded system, wireless communications, antennas and propagation,

microwave & millimeter wave technology, smart information processing, electromagnetic compatibility and environmental electromagnetics, RF/ microwave devices & materials etc.



Wenxiu Yang received M.S. degree from University of Electronic Science and Technology of China, Chengdu, China, in 2021. She is currently an assistant researcher of Southwest Jiaotong University. Her research interests include integrated circuits design, emedded system, hardware security etc.